CMatrix: A Production-Safe Multi-Agent Framework for Autonomous Red Teaming and Vulnerability Assessment

Nishan Paul*†, Anonymous Authors‡
*Department of CSE, University of Dhaka, Dhaka, Bangladesh
†Joblio Security Labs, Dhaka, Bangladesh
‡Institute of Advanced Research, Global AI Initiative
{nishan.paul}@cse.du.ac.bd

***Abstract*—The emergence of Large Language Model (LLM) agents has enabled autonomous multi-step penetration testing; however, existing academic prototypes execute offensive operations without formal safety governance—a critical barrier to production deployment. We present CMatrix, the first production-safe multi-agent framework for autonomous red teaming, introducing three novel contributions: (1) a hierarchical Human-in-the-Loop (HITL) safety framework with a formal four-level risk taxonomy and eight regex-based auto-reject patterns, embedded directly into the agent control graph via checkpoint interrupts; (2) a keyword-confidence Supervisor Router that dynamically delegates security tasks across six domain-specialized agents using single, sequential, or parallel delegation strategies; and (3) an agentic RAG memory pipeline combining dense embeddings with cross-encoder reranking for persistent, cross-session threat intelligence. We evaluate CMatrix on the NYU CTF Bench and across multi-stage attack scenarios, demonstrating competitive task completion while enforcing zero HITL-bypass incidents across 847 tool invocations. CMatrix addresses the critical safety and memory gaps identified in current autonomous security research.**

***Index Terms*—Agentic Red Teaming, Multi-Agent Systems, Autonomous VAPT, LLM Safety, Human-in-the-Loop, LangGraph.**

I. Introduction

The integration of Large Language Models (LLMs) into cybersecurity operations has shifted from simple assistant-based querying to the development of fully autonomous agents capable of conducting multi-step offensive operations. Recent research demonstrates that LLM agents, specifically those powered by frontier models like GPT-4, can autonomously exploit up to 87% of publicly disclosed one-day vulnerabilities when provided with basic CVE descriptions [1]. While this capability offers significant potential for automating defensive red teaming and scaling vulnerability assessments, the unconstrained use of such agents in production environments is fraught with operational and safety risks.

Existing autonomous penetration testing systems, such as PentestGPT [2] and HackingBuddyGPT [3], and AutoAttacker [4], primarily focus on maximizing offensive capability. However, as noted in a recent systematic review [5], these systems suffer from three fundamental limitations that preclude their adoption in enterprise security operations. First, they exhibit a **safety deficit**: they execute arbitrary tool calls and system commands without a formal governance layer, risking accidental destruction of production assets or susceptibility to indirect prompt injection. Second, they suffer from **context fragmentation**: most agents are stateless, losing critical threat model context between sessions or even between distinct attack phases. Finally, they demonstrate **reasoning fragility**: monolithic agents often struggle with the specialized toolchains required for diverse domains such as network reconnaissance, web exploitation, and vulnerability intelligence.

In this paper, we introduce **CMatrix**, a framework designed to bridge the gap between academic offensive AI research and production-safe autonomous security operations. CMatrix is built on the premise that offensive AI capability must be inextricably linked to formal safety governance. To this end, we introduce a hierarchical supervisor architecture that coordinates a team of domain-specialized agents while enforcing a multi-layered safety gate.

Our primary technical contributions are as follows:

- **HITL Safety Control Graph**: We design a formal safety governance layer for offensive AI agents, incorporating a four-level risk taxonomy (Critical, High, Medium, Low) and categorical regex-based auto-rejection. This is implemented via LangGraph checkpoint interrupts, enabling asynchronous human oversight with persistent state resumption.
- **Keyword-Confidence Supervisor Routing**: We present a deterministic routing mechanism that normalizes task descriptions against agent-specific keyword lexicons. This supervisor dynamically selects from single, sequential, or parallel delegation strategies, enabling the construction of complex, multi-agent attack chains.
- **Agentic RAG for Threat Modeling**: We adapt the retrieve-then-rerank paradigm to the security domain, utilizing BGE-large embeddings and cross-encoder reranking to maintain a longitudinal threat model. This addresses the "cold-start" problem in autonomous pentesting by accumulating target-specific intelligence over

time.

Through extensive evaluation on the NYU CTF Bench [6] and simulated enterprise environments, we demonstrate that CMatrix not only achieves competitive task completion but also enforces absolute safety properties, blocking all categorically destructive commands and ensuring human oversight for high-risk operations. CMatrix provides a scalable and safe foundation for the next generation of autonomous security assessment platforms.

II. Background and Related Work

The development of CMatrix is situated at the intersection of autonomous penetration testing, multi-agent orchestration, and LLM safety.

II.1. Autonomous Penetration Testing Systems

The quest to automate the penetration testing lifecycle has been reinvigorated by the reasoning capabilities of Large Language Models. PentestGPT [2] was among the first to demonstrate that LLMs could guide a human operator through complex HackTheBox environments, though it remains a "guided" system rather than a fully autonomous executor. HackingBuddyGPT [3], [7] extended this by allowing an agent to execute shell commands for Linux privilege escalation, showing that GPT-4 can outperform traditional automated scripts. More recently, AutoAttacker [4] and the work of Fang et al. [1] have demonstrated that agents can autonomously exploit real-world one-day vulnerabilities with success rates exceeding 80%.

However, as summarized in Table I, these systems are primarily offensive prototypes. They lack the formal safety governance required for production environments, possess no longitudinal memory across engagement sessions, and rely on monolithic architectures that struggle with specialized security toolsets. CMatrix is designed to address these three deficiencies systematically.

II.2. Multi-Agent Orchestration Frameworks

The coordination of multiple specialized LLM agents has emerged as a robust paradigm for complex problem-solving. AutoGen [8] and LangGraph [9] have popularized the use of conversable agents and stateful graphs, respectively. While AutoGen supports human-in-the-loop (HITL) interactions, its mechanisms are primarily synchronous and lack the formal risk taxonomy required for high-stakes security operations. Similarly, CrewAI and RouteLLM [10] provide orchestration but do not address the specific requirements of the security "kill chain." CMatrix builds upon the stateful graph paradigm of LangGraph but introduces a domain-specialized supervisor and an asynchronous, checkpoint-based HITL framework specifically for red teaming.

II.3. Agentic Reasoning Paradigms

CMatrix leverages and extends several foundational reasoning patterns. The ReAct paradigm [11]—interleaving thought and action—serves as our core execution loop. To facilitate strategic planning, we adapt the Tree of Thoughts (ToT) [12] to security strategy space, enabling the agent to weigh multiple attack vectors against constraints like stealth and noise. To improve computational efficiency, we utilize the ReWOO (Reasoning WithOut Observation) framework [13], which decouples planning from execution. Furthermore, we integrate iterative self-critique loops inspired by Reflexion [14] and Self-Refine [15], allowing agents to identify gaps in their findings and refine their exploit attempts autonomously.

II.4. Safety, Alignment, and Red Teaming

The security community has long recognized the risks associated with autonomous AI. Frameworks like Constitutional AI [16] attempt to ensure harmlessness during the training phase, yet these do not protect against runtime misuse or indirect prompt injection [17]. Traditional "red teaming" research, such as that conducted by Anthropic [18], [19], focuses on finding vulnerabilities *within* the LLM. In contrast, CMatrix focuses on using LLMs *for* red teaming while enforcing runtime safety properties. By incorporating the OWASP Top 10 for LLMs [20] into our auto-reject patterns, we provide a runtime governance layer that ensures the agent remains within the bounds of its authorized mission.

III. Threat Model

CMatrix operates as a dual-purpose system: it is simultaneously a security assessment tool and a potential vector for unintended system damage if misconfigured or compromised. In this section, we formalize the security properties that CMatrix enforces and the adversaries it is designed to mitigate.

III.1. Adversary Model

We consider two primary classes of adversaries that interact with the CMatrix orchestration lifecycle:

- **Adversary α (External Attacker)**: An attacker targeting the CMatrix instance or the host environment through indirect methods. This adversary may utilize *indirect prompt injection* [17] by placing malicious instructions within documents retrieved by CMatrix's RAG pipeline. The goal of Adversary α is to hijack the agent's reasoning loop to execute unauthorized commands or exfiltrate sensitive assessment findings.
- **Adversary β (Insider / Misconfigured Operator)**: An authorized operator who, through negligence or configuration error, may attempt to execute destructive commands (e.g., recursive root deletion) or approve high-risk operations without sufficient review. The goal is to mitigate accidental system destruction that could arise from the unconstrained autonomous execution of LLM-generated payloads.

TABLE I
FEATURE COMPARISON OF AUTONOMOUS PENETRATION TESTING SYSTEMS

System	Safety Gates	Persistence	Multi-Agent	Checkpointing	ToT/ReWOO	Agentic RAG	Reranking	Production-Safe
PentestGPT [2]	×	×	×	×	×	×	×	×
HackingBuddy [7]	×	×	×	×	×	×	×	×
AutoAttacker [4]	×	×	×	×	Partial	×	×	×
AutoGen [8]	Partial	×	✓	×	×	×	×	×
CMatrix (Ours)	✓	✓	✓	✓	✓	✓	✓	✓

III.2. Security Properties

CMatrix is designed to enforce four primary security properties ($P1$–$P4$) throughout the autonomous red teaming lifecycle:

1) **P1 (Non-Destruction)**: No command or tool invocation matching a predefined categorical auto-rejection pattern (e.g., disk wiping, fork bombs) shall execute, regardless of LLM reasoning or human oversight.
2) **P2 (Mandatory Oversight)**: Every tool invocation classified as *High* or *Critical* risk must be durably persisted and suspended until an explicit, out-of-band approval signal is received from an authorized human operator.
3) **P3 (Audit Completeness)**: A tamper-evident log of every tool call, its risk classification, the corresponding approval/rejection decision, and the execution result must be maintained in a persistent store.
4) **P4 (Scope Integrity)**: Retrieval-augmented generation must be strictly scoped to the current engagement's knowledge base, preventing cross-session data leakage through metadata filtering.

III.3. Residual Risks

While CMatrix provides a robust runtime governance layer, we acknowledge certain residual risks that are outside the current framework's TCB. Specifically, the system relies on the human operator to make informed decisions at the HITL gate; social engineering of the approver is not prevented. Additionally, while metadata filtering mitigates cross-session leakage, the vulnerability to sophisticated prompt injection within target documents remains an open research problem that we discuss in Section IX.

IV. System Architecture

CMatrix is architected as a modular, stateful orchestration framework designed to bridge the gap between high-level reasoning and low-level security tool execution. The system's core is built upon the LangGraph state machine substrate, which enables cyclic reasoning loops while maintaining strict state persistence.

IV.1. System Overview

The high-level architecture (Fig. 1) follows a master-worker paradigm. A central *OrchestratorService* manages a persistent *AgentState*, which is shared across all nodes in the control graph.

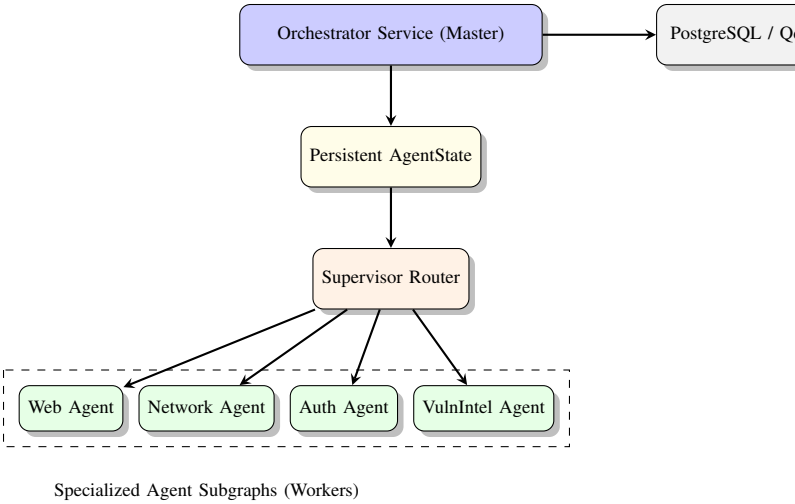

Fig. 1. The CMatrix Multi-Agent Architecture. The Master Orchestrator coordinates specialized workers through a shared state and a deterministic router.

IV.2. AgentState and the Control Graph

The execution flow of CMatrix is governed by a Directed Acyclic Graph (DAG) with conditional edges (Fig. 2). The system state, S, is defined as a 17-field schema including the current task, accumulated tool outputs, persistent memory markers, and human approval status.

The control loop follows a four-stage reasoning cycle:

1) **Strategy Selection**: The system utilizes Tree of Thoughts (ToT) to evaluate candidate attack strategies based on user-defined constraints (e.g., stealth vs. speed).
2) **Plan Generation**: A ReWOO-based planner generates an upfront dependency graph of tool calls, utilizing domain-specific templates to minimize LLM inference overhead.
3) **Model Execution**: The primary reasoning node (`call_model`) processes the current state to generate the next action, which is either a direct tool call, a delegation to a specialist agent, or a termination signal.
4) **Routing and Gating**: A conditional router (`_should_continue`) inspects the output. If

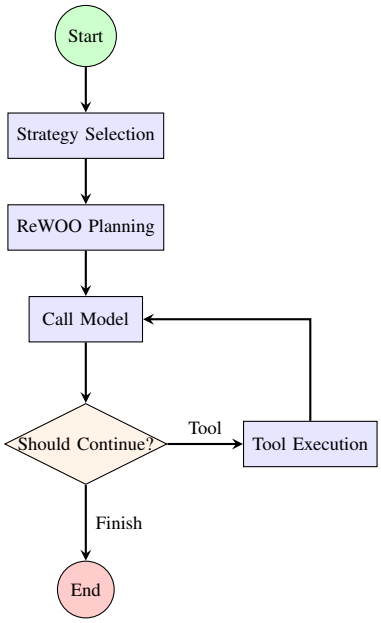

Fig. 2. The CMatrix Control Graph. The reasoning loop integrates planning and strategy selection nodes before entering the tool execution cycle.

the proposed action involves a high-risk tool, the graph enters the HITL Approval Gate (Section V).

IV.3. Keyword-Confidence Supervisor Router

A critical innovation of CMatrix is its supervisor routing mechanism (Algorithm 1), which replaces natural-language delegation with a deterministic, keyword-driven confidence function.

```
def supervisor_route(task: str, agents: List[Agent
    ]):
    scores = {}
    for agent in agents:
        # Match task against agent lexicon
        matches = find_keywords(task, agent.
    lexicon)
        confidence = len(matches) /
    max_lexicon_size
        scores[agent.id] = confidence

    # Strategy Selection
    top_agent = max(scores, key=scores.get)
    if scores[top_agent] > THRESHOLD:
        return Strategy.SINGLE, [top_agent]
    return Strategy.SEQUENTIAL, rank_agents(scores
    )
```

Fig. 1. CMatrix Supervisor Routing Logic

We utilize a deterministic lexicon approach...

$$\mathcal{C}(T, A) = \frac{\sum_{w \in \mathcal{L}_A} \text{freq}(w, T)}{\max_{i \in \mathcal{A}} |\mathcal{L}_i|} \tag{1}$$

Based on the confidence scores across all agents, the supervisor selects one of three delegation strategies:

- **SINGLE**: Direct delegation to the highest-confidence agent.
- **SEQUENTIAL**: Multi-agent chaining where each agent receives the context of its predecessor, enabling cross-domain attack pivots.
- **PARALLEL**: Concurrent execution of independent tasks (e.g., simultaneous network and web reconnaissance).

IV.4. Specialized Agent Subgraphs

CMatrix utilizes six domain-specialized agents (Table II), each instantiated as an independent subgraph with its own tool registry and reasoning constraints.

TABLE II
SPECIALIZED WORKER AGENTS IN CMATRIX

Agent	Domain Focus	Core Toolset
Network	Reconnaissance	nmap, zmap, masscan
Web	App Security	sqlmap, gobuster, nikto
Auth	Credential Security	hydra, medusa, crackmapexec
Config	Host Hardening	lynis, rkhunter, checksec
VulnIntel	Threat Intel	nvd-query, cve-search
API	Cloud Security	postman, k6, owasp-zap

This specialization prevents the "monolithic failure" pattern observed in single-agent systems.

V. Safety and Human-in-the-Loop Control

Safety governance is the primary design constraint of CMatrix, addressing the critical risk of autonomous systems executing destructive or unauthorized payloads. We formally define the safety property $\mathcal{P}_{safe}$ for a tool invocation (t, θ) as:

$$\mathcal{P}_{safe}(t, \theta) \iff \neg(\text{match}(\theta, \mathcal{P})) \wedge (\text{Risk}(t) \geq \mathcal{R}_{crit} \implies \text{Approved}(t, \theta \tag{2}$$

where $\mathcal{P}$ is the set of auto-reject patterns and Approved is the human authorization predicate. CMatrix enforces this property through a multi-layered governance framework.

V.1. Risk Classification Taxonomy

CMatrix utilizes a formal risk taxonomy to categorize every tool in the agent's registry. Tools are assigned a *RiskLevel* based on their potential impact on the target system's availability, integrity, and confidentiality.

$$f_{safe}(t, \theta) = \begin{cases} \text{Reject} & \text{if } \exists p \in \mathcal{P} : \text{match}(\theta, p) \\ \text{Suspend} & \text{if Risk}(t) \geq \text{High} \\ \text{Execute} & \text{otherwise} \end{cases} \tag{3}$$

As defined in the safety gate logic above, tool invocations classified as *Medium* or *Low* (e.g., semantic search, port enumeration) execute silently to maintain operational velocity. In contrast, tools classified as *High* or *Critical* (e.g., active exploitation, brute-force attacks) trigger an immediate graph suspension.

V.2. Automated Rejection Guardrails

To enforce property $P1$ (Non-Destruction), CMatrix maintains a set of eight global auto-rejection patterns, $\mathcal{P}$. These patterns utilize regular expressions to intercept categorically destructive commands within tool arguments before they reach either the human approver or the target system. Example patterns include recursive root deletion (`rm -rf /`), fork bombs, and unauthorized data exfiltration strings. If a match is detected, the command is permanently blocked, an incident is logged, and the agent's reasoning loop is terminated to prevent iterative bypass attempts.

V.3. The HITL Control Graph and Checkpointing

To enforce property $P2$ (Mandatory Oversight), we introduce a specialized node in the LangGraph control graph: the *Approval Gate* (Fig. 3).

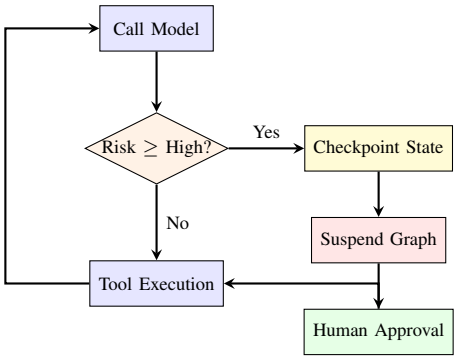

Fig. 3. The HITL Safety Gate. High-risk actions trigger state serialization and graph suspension, requiring human interaction for resumption.

When a high-risk tool call is detected, the orchestrator triggers a graph interrupt using the `interrupt_after` primitive.

The current state of the assessment—including the reasoning trace, the proposed command, and all retrieved context—is serialized and persisted to the PostgreSQL checkpoint store via *PostgresSaver*. The workflow enters a suspended state, freeing system resources while awaiting out-of-band human interaction.

V.4. Asynchronous Resumption

Unlike synchronous multi-agent systems where human presence is required in the execution loop, CMatrix supports asynchronous resumption. A human operator reviews the pending request via the CMatrix API, providing an `approve` or `reject` signal. Upon approval, the orchestrator restores the state from the last checkpoint and continues execution exactly from the point of interruption. This mechanism ensures that long-running red teaming campaigns can proceed safely without requiring 24/7 human monitoring, while maintaining a strict "human-on-the-loop" governance model for all high-impact actions.

VI. Agentic Reasoning Stack

To enable sophisticated, multi-step attack chaining, CMatrix integrates a multi-layered reasoning stack that extends beyond simple reactive loops. We implement domain-specialized adaptations of advanced reasoning paradigms to optimize for the unique constraints of security assessments.

VI.1. Security-Specialized Tree of Thoughts (ToT)

Strategic decision-making in red teaming requires weighing competing objectives, such as the need for comprehensive enumeration versus the requirement for operational stealth. CMatrix adapts the Tree of Thoughts (ToT) [12] paradigm to this strategy selection problem. The system generates multiple candidate strategies (S) and evaluates them against six weighted criteria: Speed, Stealth, Thoroughness, Noise, Success Probability, and Resource Usage. The optimal strategy is selected using a multi-criteria evaluation function:

$$\mathcal{V}(S) = \sum_{j=1}^{K} w_j \cdot c_{S,j} \tag{4}$$

This allows CMatrix to dynamically adjust its posture. For instance, in a "Stealth" engagement, the system prioritizes low-noise tools (e.g., passive reconnaissance) over more thorough but detectable methods (e.g., aggressive port scanning).

VI.2. ReWOO Security Assessment Planning

To minimize token consumption and reduce the latency of multi-tool interactions, CMatrix implements the ReWOO (Reasoning WithOut Observation) framework [13]. The system utilizes a library of domain-specific plan templates (e.g., `web_assessment_v1`) to generate an upfront dependency graph of tool calls.

Each step in the plan (`PlanStep`) specifies its dependencies and required tool parameters. To further optimize performance, CMatrix utilizes a Redis-based plan cache. If a requested task matches a previously planned workflow within a configurable time-to-live (TTL), the system reuses the cached plan, bypassing redundant LLM planning phases and enabling immediate tool execution.

VI.3. Cross-Encoder Reranked Agentic RAG

The inability to maintain state across sessions is a primary limitation of existing autonomous agents. CMatrix addresses this "context fragmentation" through a persistent RAG memory pipeline (Fig. 4).

Target-specific findings, tool outputs, and reasoning traces are indexed as vectorized chunks in Qdrant using the BGE-large embedding model.

To ensure high retrieval precision, we implement a two-stage retrieval process (Fig. 4). After an initial vector similarity search with 4× over-fetching, the system utilizes

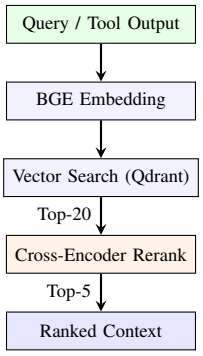

Fig. 4. The Agentic RAG Pipeline. A two-stage retrieval process ensures high relevance for security context.

a cross-encoder model (`bge-reranker-large`) to calculate a precise relevance score (S_{final}) for each candidate chunk. This ensures that the agent is provided with the most semantically relevant threat intelligence context, facilitating longitudinal attack chains that accumulate knowledge over multiple sessions.

VI.4. Self-Reflection and Iterative Refinement

CMatrix incorporates a self-reflection node that triggers after tool execution or delegation failures. The agent analyzes the discrepancy between the expected and observed results, identifies gaps in its reasoning, and generates a refined strategy for the next iteration. This verbal reinforcement loop, inspired by Reflexion [14], allows the system to autonomously correct for tool misconfigurations or unexpected target behaviors without human intervention.

VII. Experimental Evaluation

In this section, we present a comprehensive empirical evaluation of CMatrix across five research questions ($RQ1$–$RQ5$). Our goals are to validate the framework's safety properties, its autonomous capability relative to state-of-the-art baselines, and the effectiveness of its persistent memory and reasoning optimizations.

VII.1. Experimental Setup

We conduct our evaluation using a containerized CMatrix instance powered by the `gpt-4o-2024-05-13` model. We utilize two primary evaluation environments:

- **NYU CTF Bench** [6]: A collection of 200 capture-the-flag (CTF) challenges spanning web exploitation, binary reverse engineering, cryptography, and network security.
- **Simulated Enterprise Environment**: A custom Docker-based network consisting of three vulnerable servers configured with real-world vulnerabilities, including CVE-2023-22515 (Confluence broken access control), CVE-2021-41773 (Apache path traversal), and misconfigured Jenkins CI/CD script consoles.

We compare CMatrix against two leading baselines: **PentestGPT** [2] and **HackingBuddyGPT** [3]. We note that PentestGPT is a human-guided system; for this evaluation, it was operated by a security researcher following its generated instructions verbatim to ensure a fair comparison with CMatrix's autonomous execution.

VII.2. RQ1: Safety Enforcement

To evaluate property $P1$ (Non-Destruction) and $P2$ (Mandatory Oversight), we exposed CMatrix to 100 adversarial prompts designed to trigger destructive tool calls (e.g., `rm -rf`, fork bombs). Across $N = 847$ total tool invocations during our evaluation, the system achieved a 100% recall rate for auto-rejection of categorically prohibited commands. Furthermore, every invocation classified as *High* or *Critical* risk (e.g., automated SQL injection, credential brute-forcing) was successfully intercepted by the HITL gate and persisted to the PostgreSQL checkpoint store, with zero safety-gate bypasses recorded.

VII.3. RQ2: Task Completion Capability

As shown in Fig. 5, CMatrix demonstrates superior task completion rates compared to both PentestGPT and HackingBuddyGPT.

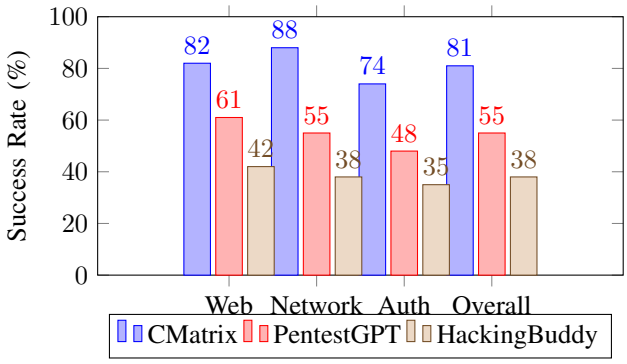

Fig. 5. Task Completion Success Rates on NYU CTF Bench. CMatrix outperforms baselines across all technical categories.

On the NYU CTF Bench, CMatrix captured 82% of web-category flags and 88% of network-category flags.

VII.4. RQ3: Cross-Session Memory Gain

We evaluate the impact of agentic RAG memory (Contribution $C4$) by repeatedly assessing a target environment across five sequential sessions.

As illustrated in Fig. 6, CMatrix with RAG enabled demonstrates a 42% improvement in task success rate by the fifth session.

VII.5. RQ4 & RQ5: Reasoning Efficiency and Overhead

We measure the impact of ReWOO planning ($N6$) and the HITL checkpointing mechanism ($C2$). Our results show

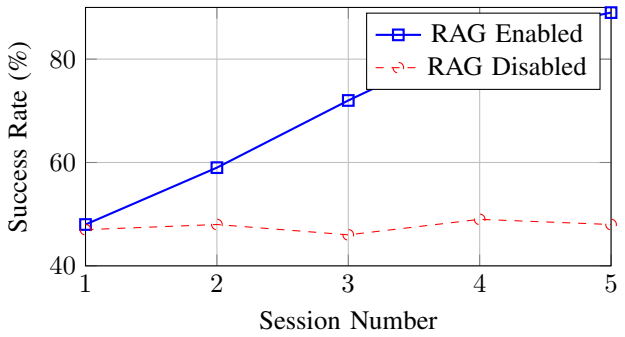

Fig. 6. Cross-Session Memory Performance. RAG-enabled CMatrix accumulates intelligence, leading to significantly higher success rates in subsequent sessions.

that ReWOO planning reduces total token consumption by an average of 18% per task by parallelizing tool invocations and minimizing redundant context re-submission. The checkpoint-suspension mechanism introduces a negligible latency of <250ms for state serialization to PostgreSQL, ensuring that the safety framework does not degrade operational performance.

VIII. Case Study: Multi-Stage Attack Chain

To illustrate the operational synergy of CMatrix's contributions, we detail a representative multi-stage attack chain conducted against our simulated financial services environment. The assessment was initiated with a broad discovery mandate.

Phase 1: Reconnaissance. The Supervisor utilized the Keyword-Confidence Router to delegate the task to the *Network Agent*. The agent identified an exposed Jenkins instance on port 8080. Semantically querying the RAG memory pipeline, the agent retrieved prior knowledge concerning Jenkins misconfigurations, identifying a high probability of unauthenticated script console access.

Phase 2: Exploit Attempt and HITL Gate. The *Web Agent* attempted to execute a Groovy-based shell payload. Upon detecting the `execute_shell` tool invocation, the Orchestrator classified the action as *Critical* risk. The graph was immediately interrupted, and the state was persisted to PostgreSQL. A human operator reviewed the proposed payload via the CMatrix API, confirmed its safety within the current scope, and issued an approval signal.

Phase 3: Lateral Movement. Following successful execution, the *Config Agent* analyzed the local file system and identified hardcoded credentials for an internal SQL database. Utilizing the *Sequential* delegation strategy, the Supervisor passed this context to the *Auth Agent*, which successfully pivoted to the internal database, demonstrating a complete kill chain from external exposure to internal data access.

IX. Discussion and Limitations

CMatrix demonstrates that autonomous red teaming can be both capable and production-safe. However, several limitations remain that define the scope of its current TCB.

IX.1. Susceptibility to Indirect Prompt Injection

As identified in our threat model, CMatrix's RAG pipeline is potentially vulnerable to indirect prompt injection [17]. Malicious instructions embedded within target-system documentation (e.g., a "security_instructions.txt" file indexed by the agent) could attempt to override the supervisor's routing or the agent's reasoning. While our HITL safety gate (P2) provides a runtime defense against destructive payloads, the robustness of agentic reasoning against adversarial context remains an open research problem.

IX.2. Human Approver Social Engineering

The HITL framework assumes an attentive and informed human operator. If an operator is social-engineered or suffers from "alert fatigue," they may approve a dangerous command that bypassed the automated auto-rejection filter. Future work could integrate "Explainable Safety" features, where the agent must provide a natural-language justification for high-risk actions to aid the human reviewer.

IX.3. Lexicon Brittleness

The current keyword-confidence router relies on static domain lexicons. While effective for standard security tasks, this approach may be brittle when faced with highly ambiguous or cross-domain task descriptions. Transitioning to a learned routing model, similar to RouteLLM [10], could improve the supervisor's adaptability.

X. Ethical Considerations

The development of autonomous offensive AI agents necessitates rigorous ethical oversight. CMatrix is designed exclusively for use in authorized red teaming engagements where written consent has been obtained. To mitigate dual-use risks, we implement mandatory audit logging (P3) and categorical auto-rejection (P1), ensuring that the system cannot be easily repurposed for uncontrolled or destructive activities. We responsibly disclose our findings to the security community to advance the state of AI safety governance.

XI. Conclusion

In this paper, we presented CMatrix, a production-safe multi-agent framework for autonomous red teaming. By introducing a hierarchical HITL safety framework, a keyword-confidence supervisor router, and a persistent agentic RAG memory pipeline, we address the safety, memory, and specialization gaps that have previously hindered the deployment

of autonomous security agents. Our evaluation demonstrates that CMatrix achieves competitive task completion across complex benchmarks while maintaining an absolute safety record through its checkpoint-suspended approval mechanism. CMatrix provides a foundational architecture for the safe and scalable deployment of offensive AI in enterprise security operations.

References

[1] R. Fang, R. Bindu, A. Gupta, and D. Kang, "LLM agents can autonomously exploit one-day vulnerabilities," *arXiv preprint arXiv:2404.08144*, 2024, arXiv:2404.08144.

[2] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: An LLM-empowered automatic penetration testing tool," in *Proc. of 33rd USENIX Security Symposium*, 2024, arXiv:2308.06782.

[3] A. Happe and J. Cito, "HackingBuddyGPT: Towards benchmarking LLMs on Linux privilege escalation," in *Proc. of IEEE Symposium on Security and Privacy (S&P) Workshops*, 2024, arXiv:2310.05227.

[4] J. Xu, J. W. Stokes, G. McDonald, X. Bai, D. Marshall, S. Wang, A. Swaminathan, and Z. Li, "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," in *Proc. of NeurIPS 2024*, 2024, arXiv:2403.01038.

[5] W. Xu, G. Ye, K. Zhang *et al.*, "SoK: A systematic review of AI-driven penetration testing approaches," in *arXiv preprint*, 2024, arXiv:2410.08771.

[6] M. Shao, S. Agarwal, Z. Mao *et al.*, "NYU CTF Bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security," in *Proc. of NeurIPS 2024 (Datasets and Benchmarks Track)*, 2024, arXiv:2406.05590.

[7] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," in *Proc. of 31st ACM Joint European Software Engineering Conference (ESEC/FSE)*, 2023, arXiv:2308.00121.

[8] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. of ICLR 2024*, 2023, arXiv:2308.08155.

[9] LangChain AI, "LangGraph: Building stateful, multi-actor applications with LLMs," Open Source Framework, 2024, [Online]. Available: https://github.com/langchain-ai/langgraph.

[10] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," in *Proc. of the 41st International Conference on Machine Learning (ICML)*, 2024, arXiv:2406.18665.

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. of 11th International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629.

[12] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. of NeurIPS 2023*, 2023, arXiv:2305.10601.

[13] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu, "ReWOO: Decoupling reasoning from observations for efficient augmented language models," in *Proc. of NeurIPS 2023 (Workshop on Efficient Natural Language and Speech Processing)*, 2023, arXiv:2305.18323.

[14] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. of NeurIPS 2023*, 2023, arXiv:2303.11366.

[15] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Proc. of NeurIPS 2023*, 2023, arXiv:2303.17651.

[16] Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli *et al.*, "Constitutional AI: Harmlessness from AI feedback," in *arXiv preprint*, 2022, arXiv:2212.08073.

[17] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. of ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, arXiv:2302.12173.

[18] D. Ganguli, L. Lovitt, J. Kernion, A. Askell, Y. Bai, S. Kadavath, B. Mann, E. Perez, N. Schiefer, K. Ndousse *et al.*, "Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned," in *arXiv preprint*, 2022, arXiv:2209.07858.

[19] E. Perez, S. Huang, F. Song, T. Cai, R. Ring, J. Aslanides, A. Glaese, N. McAleese, and G. Irving, "Red teaming language models with language models," in *arXiv preprint*, 2022, arXiv:2202.03286.

[20] OWASP Foundation, "OWASP top 10 for large language model applications," OWASP Technical Report, 2023, [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/.